Project Schedule & Work Breakdown Structure (WBS)

The timeline below incorporates the 8-day deadline extension across all project evaluation phases.

Phase & Timeline	Assessment Focus & Rubric Components	Work Breakdown Structure (WBS) Tasks	Key Deliverables & Milestones
Phase I: Project Initiation 03 Aug – 22 Aug 2026	Problem Definition, Literature Survey, Objectives, Scope, Novelty, Feasibility Study, Bill of Materials	1.1 Perform threat modeling for OWASP LLM01 (Indirect Prompt Injection). 1.2 Conduct literature review on prompt guardrails vs. structural isolation. 1.3 Assess feasibility for local SLM execution (LM Studio / Ollama). 1.4 Define Bill of Materials (BoM) for hardware/software stack.	• Finalized Capstone Abstract • Feasibility Analysis Report • Project Bill of Materials (BoM)
Phase II: Planning & Design 25 Aug – 20 Sept 2026	Methodology, System Design, WBS, Progress Review, Risk	2.1 Design Dual-LLM Quarantine Architecture & data schemas.	• System Architecture Diagram • Pydantic Schema

	Identification	2.2 Define Pydantic validation rules and Tool Execution Gateway bounds. 2.3 Identify edge cases (secondary injections, payload carrying, latency). 2.4 Draft project implementation plan & guide progress report.	Specifications • Risk Mitigation Matrix • Phase II Progress Report
Mid-Term Evaluation 22 Sept – 27 Sept 2026	Prototype Implementation, Design Validation, Experimental Results, Presentation & Viva	3.1 Demonstrate vulnerable Phase 1 prototype. 3.2 Present local LLM integration via LM Studio. 3.3 Present preliminary Dual-LLM quarantine proof-of-concept. 3.4 Defend architectural design before the evaluation committee.	• Working Baseline Prototype • Mid-Term Slide Deck & Viva • Initial Exploit Demonstration
Phase III: Implementation 29 Sept – 01 Nov 2026	Software Implementation, Coding Quality, System Integration, Testing & Validation, Performance	4.1 Implement full Dual-LLM security proxy in Python. 4.2 Integrate local inference servers	• Containerized PoC Repository • Benchmarking Dataset & Results

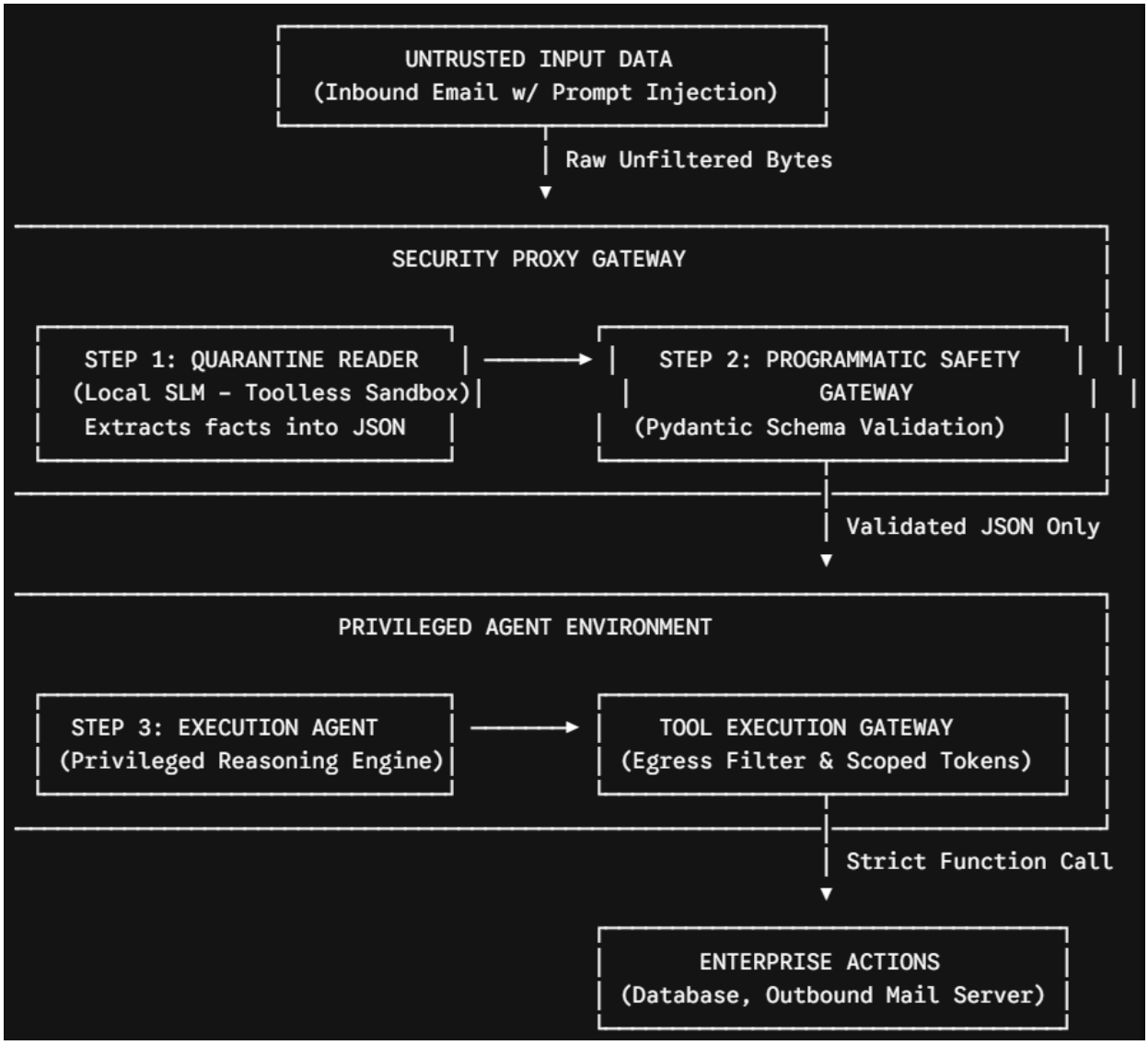
	Analysis	(LM Studio REST endpoints). 4.3 Build benchmarking engine for ASR, Latency, and Task Utility. 4.4 Write Docker Compose containerization and Deployment Runbook.	• Client Deployment Runbook • Code Quality & Unit Test Suite
Exhibition & Expert Review 03 Nov – 08 Nov 2026	Poster Presentation, Working Prototype Demonstration, Innovation Showcase	5.1 Design technical poster detailing architectural security boundaries. 5.2 Set up a live interactive attack/defense demo for domain experts. 5.3 Conduct expert walkthrough of zero-click exploit mitigation.	• Academic Project Poster • Live Interactive Demo System • Expert Feedback Documentation
Final Evaluation 09 Nov – 23 Nov 2026	Project Report, Final Presentation, Viva Voce, Research Paper / IPR Submission	6.1 Draft final academic project report adhering to institutional standards. 6.2 Finalize IEEE/ACM-style research paper on Dual-LLM isolation.	• Comprehensive Project Report • Submitted Research Paper • Final Presentation Deck

		6.3 Conduct final project defense and viva voce.	

Technical System Design

1. High-Level Architecture

The architecture enforces a **Zero-Trust Information Flow Control** pattern. Unstructured external data (emails, RAG context, API responses) is treated as inert text. It is never exposed directly to an agent equipped with tool execution privileges.



2. Module Specifications

Module A: Quarantine Reader (Step 1)

- **Purpose:** Parses untrusted raw text and extracts structured data parameters according to a target schema.
- **Execution Environment:** Sandboxed local LLM instance (LM Studio running Gemma4 12B).
- **Security Controls:**
  - Zero tool bindings attached to this model instance.
  - System prompt restricted strictly to JSON extraction tasks.
  - No access to system environment variables, local filesystem, or internal network endpoints.

Module B: Programmatic Safety Gate (Step 2)

- **Purpose:** Performs deterministic software-level validation on the JSON output emitted by the Quarantine Reader.
- **Implementation:** Python Pydantic v2 validation models combined with structural regex inspection.
- **Security Controls:**
  - **Type Enforcement:** Rejects any field containing dynamic executable instructions or non-conforming data types.
  - **Structural Stripping:** Drops unexpected key-value pairs generated by model hallucination or injection attempts.
  - **Taint Tagging:** Wraps extracted text variables in immutable data classes to prevent downstream re-injection into prompt contexts.

Module C: Tool Execution Gateway (Step 3)

- **Purpose:** Receives validated parameters and executes pre-approved enterprise actions.
- **Execution Environment:** Privileged Agent executing bound Python functions (e.g., send_email(), query_database()).
- **Security Controls:**
  - **Domain Egress Allowlisting:** Blocks outbound network calls targeting unauthorized external domains (e.g., preventing data exfiltration to evil.com).
  - **Parameter Binding:** Functions receive typed parameters as keyword arguments (kwargs), eliminating instruction-data ambiguity.

3. Data Schema Specifications

Input Quarantine Schema (Pydantic Model)

Python:

```
from pydantic import BaseModel, EmailStr, Field
from typing import Optional
class InboundEmailSchema(BaseModel):
    """Target schema enforced by Safety Gate for incoming email parsing."""
    sender_email: EmailStr
```

```
subject: str = Field(..., max_length=100)
body_summary: str = Field(..., max_length=500)
action_requested: bool = False
target_recipient: Optional[EmailStr] = None
class Config:
    extra = "forbid"  # Rejects any injected extra fields automatically
```

4. Hardware & Software Bill of Materials (BoM)

Component Type	Item Description	Function / Purpose	Cost / Availability
Compute Hardware	Development Workstation (16GB+ RAM, Dedicated GPU / Apple Silicon)	Hosting local LLMs via LM Studio	Existing Infrastructure
Inference Server	LM Studio REST Server (http://localhost:1234)	Providing local OpenAI-compatible API endpoints	Open Source (Free)
Base Models	Gemma4 12B	Local inference for quarantine & execution models	Open Weights (Free)
Runtime & Libraries	Python 3.14, Pydantic v2, FastAPI, Docker Desktop	Core application logic, validation proxy, and containerization	Open Source (Free)
Evaluation Framework	Custom Python Benchmarking Suite	Automated measurement of ASR, Latency, and Task Accuracy	Self-Developed

5. Risk Identification & Mitigation Plan

Identified Risk	Severity	Impact Area	Proposed Mitigation

Payload Carrying (Secondary Injection)	High	Executive Agent Context	Apply strict taint tracking; never re-inject string outputs from Step 1 back into unparsed system prompts.
Latency Penalty from Dual-LLM Passes	Medium	System Performance	Utilize quantized 8B models (Q4_K_M) for the quarantine reader to cap proxy overhead under 150ms.
Tool Parameter Exploitation	High	Data Exfiltration	Enforce egress domain allowlisting at the network proxy level (Step 3 Gateway).
Schema Over-Rigidity	Low	System Flexibility	Implement modular Pydantic registries to allow dynamic registration of enterprise tools.